

TKINTER

GUI WIDGETS

Overview of Tkinter

What it is: Tkinter is the standard GUI library that ships with Python, acting as a Python interface to the underlying Tcl/Tk toolkit.

Why it exists: It gives developers an easy, accessible way to build graphical applications—no command line required.

What it provides: A full set of widgets (buttons, labels, text boxes, menus), layout managers, and event-handling tools for building interactive windows.

Where it's used: Classroom demos, quick prototypes, small desktop utilities, and any Python project that needs a simple, portable interface.

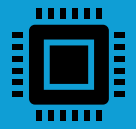
Tkinter Introduction



Tkinter is the standard GUI (Graphical User Interface) library for Python.



It provides a simple way to create windows, dialogs, events and other graphical elements in desktop applications.



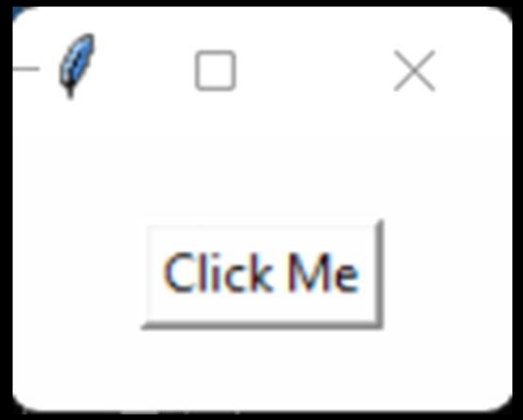
Its simplicity and accessibility make it a popular choice for beginners and for creating quick GUI prototypes.

Basic Example

```
import tkinter as tk
def on_button_click():
    print("Button clicked!")
# Create the main window
root = tk.Tk()
root.title("Simple Tkinter App")
root.geometry("100x75")
# Create a label widget
label = tk.Label(root, text="Hello, Tkinter!")
label.pack()
# Create a button widget
button = tk.Button(root, text="Click Me", command=on_button_click)
button.pack()
# Run the application
root.mainloop()
```

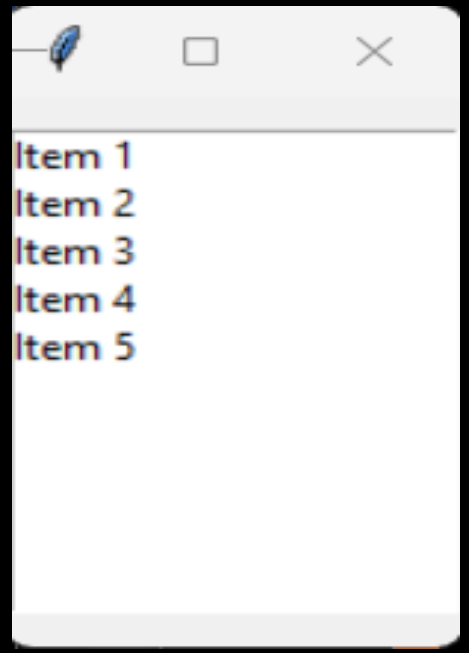


Basic Widgets



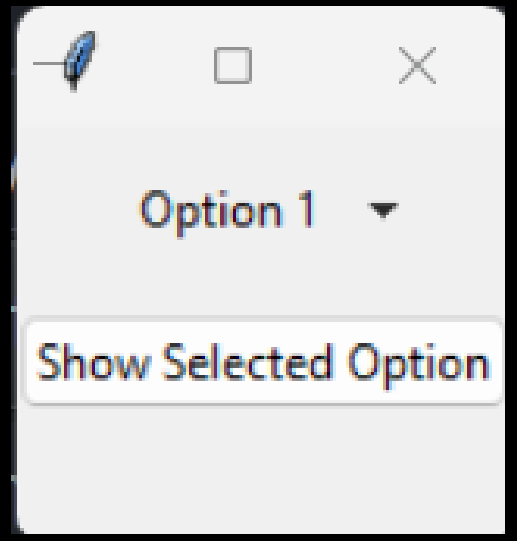
- Label: Displays text or images.
- Button: A clickable button that can trigger an action or function.
- Entry: A single-line text input field.
- Text: A multi-line text input field.
- Canvas: A widget for drawing shapes, lines, and images.
- Frame: A container for organizing and grouping other widgets.
- Toplevel: Creates a new, independent window.

Selection Widgets



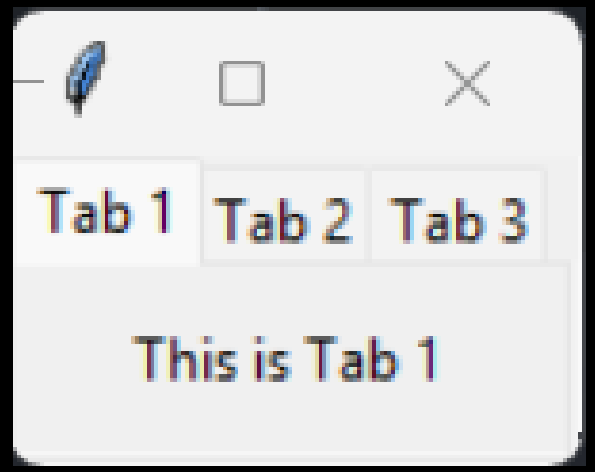
- Checkbutton: A checkbox that allows users to make multiple selections.
- Radiobutton: A button that allows users to select one option from a group.
- Listbox: A list of selectable items.
- Spinbox: A widget that allows users to select from a range of values.
- Menu: A drop-down menu that can contain various commands and submenus.
- Menubutton: A button that displays a menu when clicked.

Advanced Widgets



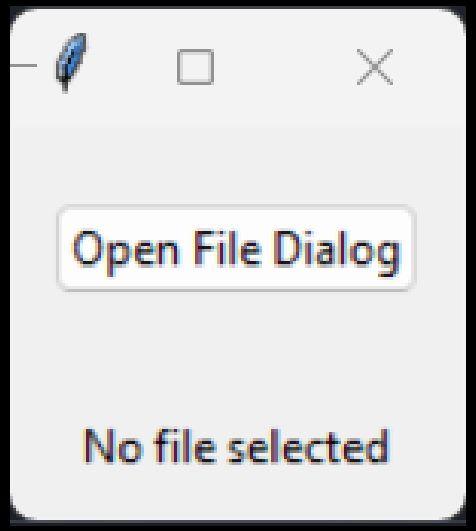
- Scale: A widget that allows users to select a value by moving a slider along a scale.
- Scrollbar: A widget that provides scrolling capability to other widgets like Text and Canvas.
- OptionMenu: A drop-down menu that allows users to select one option from a predefined list.
- Combobox (from ttk): A combination of an entry field and a drop-down list.

Layout Widgets



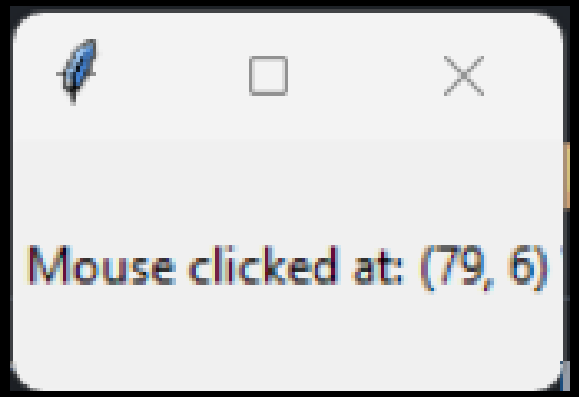
- LabelFrame: A container widget with a label, used to group related widgets.
- PanedWindow: A container widget with one or more resizable panes.
- Notebook (from ttk): A container widget that manages a collection of panes (tabbed interface).
- Style (from ttk): Used to define the style for ttk widgets.

Dialog Widgets



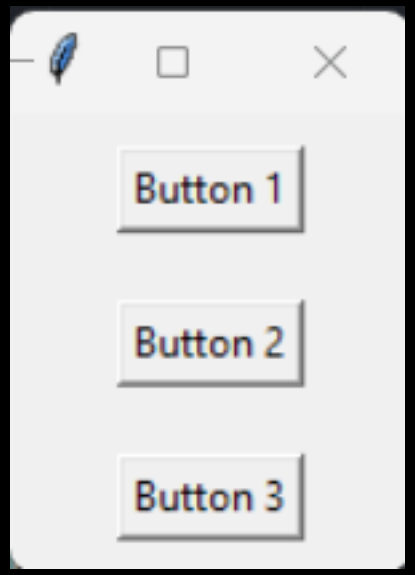
- MessageBox: A dialog that shows messages to the user (from tkinter.messagebox).
- FileDialog: Dialogs for file selection (from tkinter.filedialog).

Events



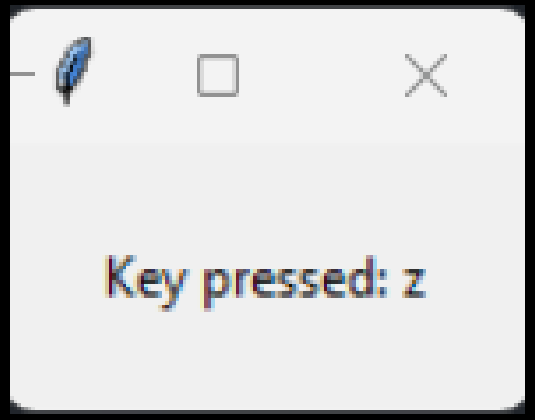
- In Tkinter, events refer to user interactions like clicking a button, pressing a key, or moving the mouse.
- Tkinter uses an event-driven programming model to handle these events, which means it waits for an event to occur and then responds by executing a callback function associated with that event.
- When an event occurs, a callback function is executed, allowing you to respond to user interactions.

Button Events



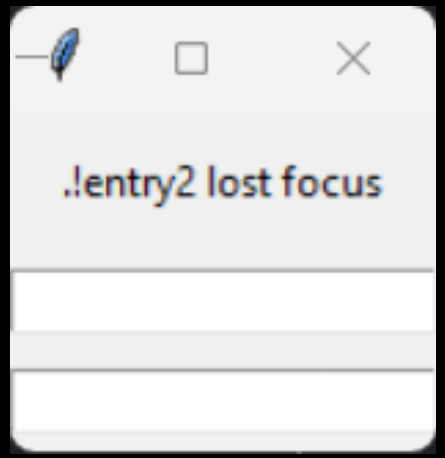
- <Button-1>: Left mouse button click
- <Button-2>: Middle mouse button click
- <Button-3>: Right mouse button click

Keyboard Events



- <KeyPress> or <Key>: Key is pressed
- <KeyRelease>: Key is released

Window Events



- <Configure>: Window is resized or moved
- <FocusIn>: Widget gains focus
- <FocusOut>: Widget loses focus

Event Binding

```
import tkinter as tk

def on_mouse_move(event):
    print(f"Mouse position: ({event.x}, {event.y})")

root = tk.Tk()
root.title("Mouse Movement Example")

# Bind the mouse motion event to the on_mouse_move function
root.bind("<Motion>", on_mouse_move)

root.mainloop()
```

Sample Output

```
Mouse position: (189, 114)
Mouse position: (185, 112)
Mouse position: (182, 111)
Mouse position: (181, 111)
Mouse position: (180, 108)
Mouse position: (177, 108)
Mouse position: (173, 105)
Mouse position: (171, 104)
Mouse position: (169, 102)
Mouse position: (164, 99)
Mouse position: (162, 99)
Mouse position: (161, 98)
Mouse position: (158, 97)
Mouse position: (156, 96)
```

Event Command

```
import tkinter as tk

def on_button_click():
    print("Button clicked using command option!")

root = tk.Tk()
root.title("Command Option Example")

button = tk.Button(root, text="Click Me", command=on_button_click)
button.pack()

root.mainloop()
```

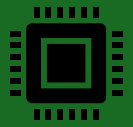
Sample output

```
Button clicked using command option!
Button clicked using command option!
Button clicked using command option!
Button clicked using command option!
Button clicked using command option!
Button clicked using command option!
```

Summary



Tkinter’s blend of simplicity, versatility, and integration with Python makes it a powerful tool for creating graphical user interfaces.



Whether you're a beginner looking to dip your toes into GUI development or an experienced developer in need of rapid prototyping,



Tkinter provides the tools and features needed to build effective and efficient applications.

References

https://www.geeksforgeeks.org/python/introduction-to-tkinter/

https://pythonguides.com/tkinter/

https://coderivers.org/blog/python-tkinter-tutorial/